Università degli Studi di Trento – DISI
Corso di Informatica Generale – a.a. 2013/2014
3° Appello - Prova scritta – 10/06/2014

Cognome.....
Nome.....
Numero Matricola.....

Completare le voci **Cognome**, **Nome** e **Matricola**, altrimenti l'elaborato **NON** verrà corretto
È **necessario** acquisire un punteggio pari a **15** nella prima parte del compito.
NON è consentito utilizzo di alcun strumento cartaceo, elettronico, informatico etc. durante la prova..
Fa parte della valutazione la leggibilità del codice C/C++.
Scrivere le risposte su questo foglio e riconsegnare. I risultati verranno pubblicati su esse3.
Tempo a disposizione: 90 minuti.

PRIMA PARTE

Punteggio: +1 risposta corretta -1 risposta errata 0 nessuna risposta

In base alle espressioni riportate dire cosa contengono le variabili x e y:

int x = 7, y= 4; x = y + (y += x); x [ ] y [ ]
int x = 6, y= 3; x = y + (x -= x); x [ ] y [ ]
int x = 5, y= 2; y = (x -= y) +x; x [ ] y [ ]

Calcolare la somma tra $(1101)_2$ e $(1011)_2$?

SI ☐ NO ☐ $(11000)_2$
SI ☐ NO ☐ $(110)_4$
SI ☐ NO ☐ $(32)_8$
SI ☐ NO ☐ $(1B)_{16}$

Relativamente alla dichiarazione di un array monodimensionale (vettore):

SI ☐ NO ☐ Il vettore è un insieme di elementi non omogenei tra loro.
SI ☐ NO ☐ Il contenuto di un elemento del vettore non puo' essere modificato dopo la sua inizializzazione
SI ☐ NO ☐ Con il vettore possiamo indicare tanti dati dello stesso tipo con un solo nome collettivo di variabile: l'identificatore del vettore.
SI ☐ NO ☐ Gli elementi di un vettore si distinguono uno dall'altro attraverso l'indice che viene assegnato nel vettore.

Cosa contengono x, y, z dopo l'esecuzione delle seguenti istruzioni

```
int x = 4, y = 3, z;
if (--x > y++) { z = x % 2;} else { z = x * y; }
```

x [ ] y [ ] z [ ]

Data la seguente dichiarazione di variabili int alfa = 4; int beta = 7; int *pointer;
dire cosa stampano le seguenti istruzioni (eseguite in ordine):

pointer = &alfa; printf("alfa -> %d, beta -> %d, pointer -> %d", alfa, beta, *pointer);	
beta = *pointer; printf("alfa -> %d, beta -> %d, pointer -> %d", alfa, beta, *pointer);	
alfa = *pointer; printf("alfa -> %d, beta -> %d, pointer -> %d", alfa, beta, *pointer);	
*pointer = 5; printf("alfa -> %d, beta -> %d, pointer -> %d", alfa, beta, *pointer);	

SECONDA PARTE

[Domanda 1]

a) Definire in modo più preciso possibile cosa si intende per albero BST

b) Applicare l'algoritmo di riempimento di un albero BST a questa sequenza di valori interi: disegnare l'albero così ottenuto
valori uguali DEVONO essere inseriti nel ramo di destra.

5	0	5	2	2	1	5	6

c) Per l'albero così realizzato inoltre calcolare

Cammino							
Altezza							
Visita pre-ordine							
Visita post-ordine							
Visita in-ordine							
Visita level-ordine							

[Domanda 2]

a) dare la definizione di funzione ricorsiva

b) creare una funzione ricorsiva che permetta il calcolo della potenza a^b mediante ricorsione

$$\text{potenza}(a,b) := \begin{cases} 1 & \text{if } b = 0 \\ a \cdot \text{potenza}(a, b-1) & \text{otherwise} \end{cases}$$

c) descrivere l'esecuzione della funzione (le chiamate effettuate) in seguito all'invocazione: potenza(3,3).

[Domanda 3] Un bambino deve salire una scala di N gradini.
Può compiere passi da 1, 2 o 3 gradini. Descrivere un algoritmo che calcoli in quanti possibili modi il bambino può salire le scale. (Suggerimento: Usare formulazione ricorsiva)

[Domanda 4] Risolvere la seguente equazione di ricorrenza: $\begin{cases} T(1) = 1 \\ T(n) = 2(T/2) + n \end{cases}$

[Domanda 5] data la seguente funzione ricorsiva

```
int minimo(int a[], int n) {
    printf("n=%d\n",n);
    if(n==0) return a[0];
    if (a[n] < minimo(a, n-1) ) {
      return a[n];
    }else{
      return minimo(a, n-1);
    }
}
int a[] = {7, 4, 2, 5};
```

dire cosa vien stampato con questo comando `printf("%d",minimo(a,3));`

[Domanda 6]

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 6
typedef int Matricola[MAX];
Matricola m;

int f3(int p) {
   return p/2;
}
int f1(int p, int *q) {
   int x = 2;
   x = x + (*q);     printf("X=%d\n", x);
   p = (*q + x);     printf("P=%d\n", p);
   *q = f3(p)%MAX;   printf("Q=%d\n", *q);
   return *q;
}
int f2() {
  int y;
  y = m[3]%MAX;     printf("Y= %d\n", y);
  y = f1(m[y],&y);  printf("Y= %d\n", y);
  printf("m[Y]= %d\n", m[y]);
  return m[y]-y;
}
main() {
  // Inserisci le cifre della tua matricola, una alla volta
  for (int i=0 ; i<MAX ; i++) { scanf("%d", &m[i]); }
  printf("Output: %d\n", f2());
}
```

MATRICOLA ☐☐☐☐☐☐

Si scriva esattamente l'output (istruzioni ***printf***), nel riquadro a fianco, che viene prodotto durante l'esecuzione del programma.